Django REST Framework-এ **APIView** use করলে URL থেকে view call করার **মূলত ২টা standard উপায়** আছে। নিচে step-by-step, example সহ বুঝাচ্ছি 👇

1️⃣ `APIView.as_view()` ব্যবহার করে (সবচেয়ে common)

👉 **APIView হলো class-based view**, তাই URL-এ call করার সময় অবশ্যই `as_view()` ব্যবহার করতে হবে।

📌 View (views.py)

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

class HelloAPIView(APIView):
    def get(self, request):
        return Response({"msg": "Hello GET API"})

    def post(self, request):
        return Response({"msg": "Hello POST API"},
status=status.HTTP_201_CREATED)
```

📌 URL (urls.py)

```
from django.urls import path
from .views import HelloAPIView

urlpatterns = [
    path('hello/', HelloAPIView.as_view()),
]
```

🔍 Call

- GET → `/hello/`
- POST → `/hello/`

✅ **APIView হলে `as_view()` mandatory**

2️⃣ HTTP method অনুযায়ী automatically call হয়

APIView নিজে থেকেই HTTP method অনুযায়ী function call করে দেয়:

HTTP Method	View Method
GET	`get()`
POST	`post()`
PUT	`put()`
PATCH	`patch()`
DELETE	`delete()`

👉 URL-এ আলাদা করে কিছু করার দরকার নাই
👉 Browser/Postman যেই method পাঠাবে, সেই method call হবে

❌ যেভাবে call করা যাবে না

```
# ❌ ভুল
path('hello/', HelloAPIView)
```

কারণ:

- APIView = class
- Django URL expects = callable function
- `as_view()` এই class-টাকে callable বানায়

🔁 APIView vs api_view decorator (Important difference)

✔️ `@api_view` → function-based

```
from rest_framework.decorators import api_view
from rest_framework.response import Response

@api_view(['GET'])
def hello_fn(request):
    return Response({"msg": "Hello Function API"})
```

```
path('hello/', hello_fn)   # as_view লাগবে না
```

✔️ `APIView` → class-based

```
path('hello/', HelloAPIView.as_view())
```

🔑 Summary (Interview friendly)

Case	as_view() লাগবে?
APIView	✅ হ্যাঁ
GenericAPIView	✅ হ্যাঁ
ViewSet	⚠️ Router হলে না
@api_view	❌ না

🧠 কখন APIView ব্যবহার করবো?

- Custom logic বেশি
- Full control দরকার
- CRUD পুরোপুরি standard না
- Authentication / permission নিজে handle করতে চাইলে

Router use করলে APIView সরাসরি কাজ করে না — এটা খুবই important concept।

চল step-by-step পরিষ্কার করি 👇

❌ Router দিয়ে APIView use করা যায় না

```
class MyAPIView(APIView):
    ...
```

```
router.register('test', MyAPIView)  # ❌ এটা ভুল
```

🔴 কারণ:

- **Router কাজ করে শুধু ViewSet / GenericViewSet-এর সাথে**
- APIView-এর মধ্যে `list()`, `retrieve()` এর মতো action নাই

✅ Router use করলে কী use করতে হবে?

👉 **ViewSet family**

Router	Use with
DefaultRouter	ViewSet
SimpleRouter	ModelViewSet
Router	ReadOnlyModelViewSet
❌ Router	APIView

1️⃣ Basic ViewSet + Router

📌 views.py

```
from rest_framework.viewsets import ViewSet
from rest_framework.response import Response

class HelloViewSet(ViewSet):

    def list(self, request):
        return Response({"msg": "GET all data"})

    def retrieve(self, request, pk=None):
        return Response({"msg": f"GET single data {pk}"})

    def create(self, request):
        return Response({"msg": "POST create"})
```

📌 urls.py

```
from rest_framework.routers import DefaultRouter
from .views import HelloViewSet

router = DefaultRouter()
router.register('hello', HelloViewSet, basename='hello')

urlpatterns = router.urls
```

🔍 Generated URLs

URL	Method	Method Call
/hello/	GET	list()
/hello/	POST	create()
/hello/1/	GET	retrieve()
/hello/1/	PUT	update()
/hello/1/	DELETE	destroy()

2️⃣ ModelViewSet (Most common in real projects)

```
from rest_framework.viewsets import ModelViewSet
from .models import Student
from .serializers import StudentSerializer

class StudentViewSet(ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
```

```
router.register('students', StudentViewSet)
```

🔥 Automatic CRUD ready

3️⃣ Router + Custom action (`@action`)

```
from rest_framework.decorators import action

class StudentViewSet(ModelViewSet):

    @action(detail=False, methods=['get'])
    def active(self, request):
        return Response({"msg": "Active students"})
```

🔗 URL:

```
/students/active/
```

🧠 Router vs APIView (clear difference)

Feature	APIView	Router + ViewSet
Manual URL mapping	✅	❌
Auto CRUD	❌	✅
Router support	❌	✅
Clean URL	❌	✅
Large project	❌	✅

🔑 Rule to remember (Interview line)

> **Router use করলে অবশ্যই ViewSet use করতে হবে, APIView নয়**

🔥 Real project suggestion (তোমার scholarship system context◌)

- Simple/custom endpoint → `APIView`
- CRUD-heavy modules → `ModelViewSet + Router`

---

full example for APIView

চল **APIView + Serializer + Authentication + Permission** একসাথে দিয়ে একটা **mini TODO app (production-style)** বানাই।
আমি ধীরে ধীরে explain করবো, যেন তুমি concept + flow দুটোই বুঝতে পারো 👍

🧩 TODO App (APIView based)

Feature

- User login থাকতে হবে
- User নিজের TODO দেখবে / তৈরি করবে
- অন্য user-এর data access করা যাবে না
- Token Authentication

1️⃣ Model (models.py)

```
from django.db import models
from django.contrib.auth.models import User

class Todo(models.Model):
    user = models.ForeignKey(
        User,
        on_delete=models.CASCADE,
        related_name='todos'
    )
    title = models.CharField(max_length=200)
    is_completed = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

```
python manage.py makemigrations
python manage.py migrate
```

2️⃣ Serializer (serializers.py)

```
from rest_framework import serializers
from .models import Todo

class TodoSerializer(serializers.ModelSerializer):
    user = serializers.ReadOnlyField(source='user.username')

    class Meta:
        model = Todo
        fields = ['id', 'user', 'title', 'is_completed', 'created_at']
```

📌 `ReadOnlyField` → user client থেকে পাঠাতে পারবে না

3️⃣ Authentication setup (settings.py)

```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'rest_framework.authtoken',
```

```
    'todo',
]
```

```
python manage.py migrate
```

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}
```

4️⃣ Token generate API (login)

urls.py (project or app)

```
from rest_framework.authtoken.views import obtain_auth_token
from django.urls import path

urlpatterns = [
    path('api/token/', obtain_auth_token),
]
```

📌 POST:

```
{
  "username": "admin",
  "password": "1234"
}
```

Response:

```
{
  "token": "a1b2c3d4..."
}
```

5️⃣ APIView (views.py)

📌 Todo List + Create API

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from rest_framework.permissions import IsAuthenticated

from .models import Todo
from .serializers import TodoSerializer

class TodoListCreateAPIView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        todos = Todo.objects.filter(user=request.user)
        serializer = TodoSerializer(todos, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = TodoSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save(user=request.user)
            return Response(serializer.data,
status=status.HTTP_201_CREATED)
        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)
```

📌 Todo Detail API (Update + Delete)

```
from django.shortcuts import get_object_or_404

class TodoDetailAPIView(APIView):
    permission_classes = [IsAuthenticated]

    def get_object(self, pk, user):
        return get_object_or_404(Todo, pk=pk, user=user)

    def get(self, request, pk):
        todo = self.get_object(pk, request.user)
        serializer = TodoSerializer(todo)
        return Response(serializer.data)

    def put(self, request, pk):
        todo = self.get_object(pk, request.user)
        serializer = TodoSerializer(todo, data=request.data)
        if serializer.is_valid():
```

```
            serializer.save(user=request.user)
            return Response(serializer.data)
        return Response(serializer.errors, status=400)

    def delete(self, request, pk):
        todo = self.get_object(pk, request.user)
        todo.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

🔐 এখানে permission + ownership দুটোই enforced

6️⃣ URLs (urls.py)

```
from django.urls import path
from .views import TodoListCreateAPIView, TodoDetailAPIView

urlpatterns = [
    path('todos/', TodoListCreateAPIView.as_view()),
    path('todos/<int:pk>/', TodoDetailAPIView.as_view()),
]
```

7️⃣ Request flow (Postman / Frontend)

Header (mandatory)

```
Authorization: Token a1b2c3d4...
```

POST `/todos/`

```
{
  "title": "Learn APIView",
  "is_completed": false
}
```

🔄 Complete Flow Diagram

```
Client
  |
  |-- Token (login)
```

```
    |
    |-- Request + Token
          |
          |--> Authentication (TokenAuthentication)
          |--> Permission (IsAuthenticated)
          |--> APIView
          |--> Serializer (validate)
          |--> Model
          |--> Response
```

🧠 Why APIView here?

✔ Full control
✔ Ownership logic easy
✔ Interview-friendly
✔ Custom behavior possible

❗ Interview One-liner

> **APIView gives full control over request-response cycle, unlike ViewSet which abstracts it**

চাও তো আমি পরের ধাপে দেখাতে পারি 👇
👉 **Same TODO app using ModelViewSet + Router**
👉 **Custom Permission class (IsOwner)**
👉 **APIView + JWT authentication version**

যেটা চাই, বললেই হবে 😊

তুমি basically জানতে চাও — **এই `TodoDetailAPIView` -এ সব HTTP method (CRUD + extra operations) কিভাবে add করা যায় (industry-level full version)।**

আমি clean করে **full production-ready version + explanation** দিচ্ছি 👇

✅ ✅ Full TodoDetailAPIView (All Operations Included)

```python
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from rest_framework import status
from django.shortcuts import get_object_or_404

class TodoDetailAPIView(APIView):
    permission_classes = [IsAuthenticated]

    def get_object(self, pk, user):
        return get_object_or_404(Todo, pk=pk, user=user)

    # ✅ READ (Single Todo)
    def get(self, request, pk):
        todo = self.get_object(pk, request.user)
        serializer = TodoSerializer(todo)
        return Response(serializer.data)

    # ✅ FULL UPDATE (PUT)
    def put(self, request, pk):
        todo = self.get_object(pk, request.user)
        serializer = TodoSerializer(todo, data=request.data)

        if serializer.is_valid():
            serializer.save(user=request.user)
            return Response(serializer.data)

        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)

    # ✅ PARTIAL UPDATE (PATCH)
    def patch(self, request, pk):
        todo = self.get_object(pk, request.user)
        serializer = TodoSerializer(todo, data=request.data, partial=True)

        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)

        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)

    # ✅ DELETE
    def delete(self, request, pk):
        todo = self.get_object(pk, request.user)
        todo.delete()
        return Response(
            {"message": "Todo deleted successfully"},
            status=status.HTTP_204_NO_CONTENT
```

```
        )

    # ✅ OPTIONAL: Custom Action (e.g., mark complete)
    def post(self, request, pk):
        """
        Example: Mark todo as completed
        """
        todo = self.get_object(pk, request.user)
        todo.completed = True
        todo.save()

        return Response({
            "message": "Todo marked as completed"
        }, status=status.HTTP_200_OK)
```

🔥 Breakdown (Interview Level Understanding)

✅ 1. `get_object()`

```
return get_object_or_404(Todo, pk=pk, user=user)
```

👉 Important:

- শুধু নিজের todo access করতে পারবে
- Security layer (VERY IMPORTANT)

✅ 2. GET → Read

```
GET /api/todos/1/
```

👉 Single todo fetch

✅ 3. PUT → Full Update

```
PUT /api/todos/1/
```

👉 সব field send করতে হবে
👉 Missing field = overwrite

✅ 4. PATCH → Partial Update ⭐ (VERY IMPORTANT)

```
PATCH /api/todos/1/
```

👉 শুধু change হওয়া field পাঠালেই হবে

Example:

```
{
  "title": "Updated title"
}
```

✅ 5. DELETE

```
DELETE /api/todos/1/
```

👉 record delete

✅ 6. POST (Custom Action)

```
POST /api/todos/1/
```

👉 Custom logic:

- mark complete
- archive
- toggle status

⚠️ Important Note (Real Industry)

👉 Normally:

- `POST` used for create
- Detail view এ `POST` use করা uncommon

Better approach 👇

🔥 Better Practice (Recommended)

👉 Custom action use করো:

Option 1: Separate endpoint

```
POST /api/todos/1/complete/
```

Option 2: ViewSet (BEST PRACTICE)

```
from rest_framework.decorators import action

class TodoViewSet(ModelViewSet):

    @action(detail=True, methods=['post'])
    def complete(self, request, pk=None):
        todo = self.get_object()
        todo.completed = True
        todo.save()

        return Response({"message": "Completed"})
```

🚀 Pro Tips (Very Important)

✅ 1. Validation Error Always Standardize

```
return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

✅ 2. Use status constants

❌ bad:

```
status=400
```

✅ good:

```
status=status.HTTP_400_BAD_REQUEST
```

✅ 3. Always secure queryset

```
user=request.user
```

✅ 4. Prefer ViewSet in real projects

👉 Less code
👉 Router auto endpoint
👉 Clean architecture

🎯 Final Summary

তুমি এখন এই API দিয়ে করতে পারো:

Method	কাজ
GET	single todo
PUT	full update
PATCH	partial update
DELETE	delete
POST	custom action

👉 চাইলে আমি next step এ তোমাকে দেখাতে পারি:

- 🔥 APIView vs GenericAPIView vs ViewSet (deep comparison)
- 🔥 Same Todo system using ModelViewSet (industry level clean version)
- 🔥 Router + URL structure complete setup

বললেই আমি full production architecture বানিয়ে দিব 🚀

তুমি এখন "**all todo**" বলতে চাচ্ছো — মানে:

👉 **সব Todo list দেখানোর API (List API)**

আমি তোমাকে 3টা level এ দেখাচ্ছি 👇
(Interview + Real project ready)

✅ ✅ 1. Simple APIView (All Todos)

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated

class TodoListAPIView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        todos = Todo.objects.filter(user=request.user)
        serializer = TodoSerializer(todos, many=True)
        return Response(serializer.data)
```

🔥 Endpoint

```
GET /api/todos/
```

👉 Logged-in user এর সব todo return করবে

✅ ✅ 2. Create + List (Best for APIView)

```
class TodoListCreateAPIView(APIView):
    permission_classes = [IsAuthenticated]

    # ✅ GET → All Todos
    def get(self, request):
        todos = Todo.objects.filter(user=request.user)
        serializer = TodoSerializer(todos, many=True)
        return Response(serializer.data)

    # ✅ POST → Create Todo
    def post(self, request):
        serializer = TodoSerializer(data=request.data)

        if serializer.is_valid():
            serializer.save(user=request.user)
            return Response(serializer.data, status=201)

        return Response(serializer.errors, status=400)
```

🔥 Endpoint

Method	URL	কাজ
GET	`/api/todos/`	সব todo
POST	`/api/todos/`	create todo

🚀 Example Response

```
[
  {
    "id": 1,
    "title": "Learn Django",
    "completed": false
  },
  {
    "id": 2,
    "title": "Build API",
    "completed": true
  }
]
```

🔥 Advanced (Filtering + Search) ⭐

```
def get(self, request):
    todos = Todo.objects.filter(user=request.user)

    # filter by completed
    completed = request.query_params.get('completed')
    if completed is not None:
        todos = todos.filter(completed=completed)

    serializer = TodoSerializer(todos, many=True)
    return Response(serializer.data)
```

🔥 Example ব্যবহার

```
GET /api/todos/?completed=true
```

👉 শুধু completed todos

💡 Pagination (Very Important)

```
from rest_framework.pagination import PageNumberPagination

class TodoPagination(PageNumberPagination):
    page_size = 5
```

```
def get(self, request):
    todos = Todo.objects.filter(user=request.user)
    paginator = TodoPagination()
    paginated_todos = paginator.paginate_queryset(todos, request)

    serializer = TodoSerializer(paginated_todos, many=True)
    return paginator.get_paginated_response(serializer.data)
```

🔥 BEST PRACTICE (Industry) ⭐

👉 APIView না, এটা use করো:

```
from rest_framework.generics import ListCreateAPIView

class TodoListCreateAPIView(ListCreateAPIView):
    serializer_class = TodoSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return Todo.objects.filter(user=self.request.user)

    def perform_create(self, serializer):
        serializer.save(user=self.request.user)
```

🎯 Final বুঝে রাখো

Approach	কখন use করবে
APIView	learning / custom logic
GenericAPIView	medium level

Approach	কখন use করবে
ViewSet	production ⭐

👉 চাইলে next step এ আমি তোমাকে দেখাতে পারি:

- 🔥 Full Todo API using ModelViewSet (1 class = full CRUD)
- 🔥 Router setup (auto endpoints)
- 🔥 Swagger API documentation

বললেই complete backend structure বানিয়ে দিব 🚀
না — **index-এর মধ্যে field name গুলো অবশ্যই model-এর existing field হতে হবে**। তুমি নিজের ইচ্ছামতো নতুন নাম দিতে পারবে না ❌

✅ Correct ধারণা

```
class Meta:
    indexes = [
        models.Index(fields=["user", "completed"]),
    ]
```

👉 এখানে:

- `"user"` → model-এর field ✔️
- `"completed"` → model-এর field ✔️

❌ ভুল example

```
models.Index(fields=["username", "status"])
```

👉 যদি তোমার model-এ:

- `username` না থাকে ❌
- `status` না থাকে ❌

👉 তাহলে error দিবে (migration time-এ)

🔥 তুমি যে model বানাইছো তার valid index example

```
class Meta:
    indexes = [
        models.Index(fields=["user"]),
        models.Index(fields=["completed"]),
        models.Index(fields=["user", "completed"]),
        models.Index(fields=["priority"]),
    ]
```

⚡ Index কেন ব্যবহার করি?

👉 Database search fast করার জন্য

Example:

```
Todo.objects.filter(user=request.user, completed=True)
```

👉 এই query fast হবে যদি index থাকে:

```
["user", "completed"]
```

🔥 Advanced (Interview Level)

✅ Composite Index

```
models.Index(fields=["user", "completed"])
```

👉 multiple field একসাথে optimize করে

✅ Ordering matters ⚠️

```
models.Index(fields=["user", "completed"])  # ✔️
```

👉 works best for:

```
filter(user=..., completed=...)
```

কিন্তু:

```
models.Index(fields=["completed", "user"])
```

👉 different behavior দিতে পারে

🎯 Final Summary

প্রশ্ন	উত্তর
index এ custom নাম দেওয়া যাবে?	❌ না
কি দিতে হবে?	✔️ model-এর existing field
multiple field দেওয়া যাবে?	✔️ হ্যাঁ
order important?	⚠️ হ্যাঁ

👉 চাইলে আমি তোমাকে next level এ শেখাতে পারি:

- 🔥 `db_index=True` vs `Meta.indexes` difference
- 🔥 Query optimization (Django ORM deep dive)
- 🔥 Real project performance tuning

বললেই deep dive শুরু করি 🚀